

Using Source Code Data Structures to Detect Vulnerable Code Units with Deep Graph Convolutional Neural Networks

1st José D’Abruzzo Pereira
University of Coimbra
CISUC/LASI, DEI
Coimbra, Portugal
josep@dei.uc.pt

2nd Luís Vieira
University of Coimbra
CISUC/LASI, DEI
Coimbra, Portugal
luisvieira@student.dei.uc.pt

3rd Nuno Lourenço
University of Coimbra
CISUC/LASI, DEI
Coimbra, Portugal
naml@dei.uc.pt

Abstract—Software vulnerabilities pose risks to system security. Traditional detection methods have important limitations: static analysis relies on expert-crafted rules and often yields high false-positive rates, while dynamic analysis suffers from incomplete coverage and high computational cost. Machine Learning (ML) and Deep Learning (DL) provide alternatives by learning vulnerability patterns from data, with graph-based approaches being particularly promising due to their ability to capture structural and semantic properties of code. This work evaluates Deep Graph Convolutional Neural Networks (DGCNNs) on multiple graph representations of C functions for vulnerability detection. We extend a graph extraction pipeline for Control Flow Graphs (CFGs), Abstract Syntax Trees (ASTs), and Program Dependence Graphs (PDGs) using a Linux kernel vulnerability dataset, and implement a modular neural framework with a DGCNN-based classifier and an Autoencoder (AE) for pretraining. Results show that CFGs achieve the most stable performance, especially when combined with a lightweight 1D CNN and pretraining, while ASTs and PDGs yield less stable embeddings. While absolute F1-scores reflect the inherent difficulty of detecting sparse security signals in large-scale repositories, the results confirm the feasibility of applying DGCNNs to highly imbalanced real-world datasets. We demonstrate that our framework achieves stable performance and measurable gains over previous baselines, particularly when leveraging AE pretraining to structure the latent space of CFGs. These findings highlight that in production-scale code analysis, representation quality and embedding stability are more critical than increasing classifier complexity.

Index Terms—Software Security, Machine Learning, Vulnerability Detection, Deep Graph Convolutional Neural Networks, Static Code Analysis.

I. INTRODUCTION

Today’s society relies heavily on software-driven systems, which increases exposure to software vulnerabilities. These issues remain a persistent concern, as they threaten the security of systems, data, and critical infrastructure. Recent reports show a significant rise in exploitation: according to the 2024 Verizon Data Breach Investigations Report, attacks leveraging vulnerabilities as the primary breach vector grew by 180% since 2024 [1]. This trend highlights the limitations of current detection and mitigation practices and reinforces the need for more effective techniques.

Traditional approaches to vulnerability detection, such as static and dynamic analysis, have long been essential to secure software development [2]. Static analysis examines code or binaries without execution and is effective at identifying common issues through techniques like data flow and control flow analysis [2]. However, Static Analysis Tools (SATs) rely on expert-crafted rules, which are labor-intensive to maintain and less adaptable to new or diverse vulnerability types, and they often produce high False Positive (FP) rates, which burden developers. Dynamic analysis evaluates program behavior at runtime and can reveal vulnerabilities related to input validation, access control, and error handling [2]. Yet, dynamic techniques cannot guarantee full coverage because only executed paths are analyzed, leading to missed vulnerabilities, increased computational cost, and difficulty simulating real-world environments [3].

Given these limitations, alternative approaches capable of generalizing to unseen patterns have received growing attention. Machine Learning (ML) and Deep Learning (DL) offer promising solutions by learning directly from data rather than relying solely on handcrafted rules. Classical ML algorithms such as Support Vector Machines (SVMs), k-Nearest Neighbors (kNN), Random Forests (RFs), and Decision Trees (DTs) have been used to classify vulnerable code units [4]. More recently, Neural Networks (NNs), including Convolutional Neural Networks (CNNs) and Graph Neural Networks (GNNs), have demonstrated the ability to capture structural and contextual information in source code [5]. GNNs, in particular, can model relationships encoded in graph representations, making them highly suitable for learning the complex patterns that characterize software vulnerabilities [2].

Graph-based learning has advanced substantially, enabling the use of increasingly expressive architectures to analyze source code. Graph Convolutional Networks (GCNs) established effective mechanisms for processing graph-structured data [6], while Deep Graph Convolutional Neural Networks (DGCNNs) introduced deeper hierarchical feature extraction. Yan *et al.* [7] proposed MAGIC, which applied DGCNNs to malware classification using Control Flow Graphs (CFGs),

demonstrating strong predictive performance and robustness. Inspired by this, *Pereira et al.* [8] adapted DGCNNs to detect vulnerable C functions using CFGs enriched with memory-related features. Their work achieved high recall but suffered from many FPs and remained limited to a single graph representation and a fixed classifier, raising questions about whether alternative graph structures, classifier architectures, or pretraining strategies could improve performance.

Building on that work, this paper extends the study by incorporating additional graph representations (Program Dependence Graphs (PDGs) and Abstract Syntax Trees (ASTs)), comparing lightweight and deep classifiers, and exploring the impact of Autoencoder (AE) pretraining. Graphs are extracted with the tool Joern [9], enabling the capture of syntactic, semantic, and dependency-based properties of C code. We developed a modular DL framework that includes multiple classifier branches, embedding analysis tools, and pretraining capabilities. This work evaluates how graph representation, classifier complexity, pretraining, class imbalance, and graph size influence predictive performance and embedding stability. Hence, we address the following Research Questions (RQs):

- **RQ1:** To what extent can new techniques (*e.g.*, AE pretraining, different classifiers, data balancing strategies) improve the results previously obtained with CFGs?
- **RQ2:** Can ASTs and PDGs outperform or complement CFGs in the detection of vulnerable functions?
- **RQ3:** Does the use of AE pretraining improve the DGCNN embeddings and lead to better classification results?
- **RQ4:** How does classifier complexity (lightweight Conv1D vs. deeper Very Deep Convolutional Network (VGG)) affect stability and performance of vulnerability detection?

This work focuses on C code due to its low-level nature and lack of built-in memory safety, which make it especially prone to vulnerabilities such as buffer overflows, use-after-free errors, and pointer misuse [10], [11]. To evaluate the proposed framework, we use a dataset of C functions extracted from the Linux kernel project [12]. Unlike many contemporary studies that evaluate models on balanced synthetic datasets to achieve high metrics, this work evaluates the proposed framework on a highly imbalanced subset of the Linux kernel. With a vulnerable class ratio of only 2.6%, the dataset represents the ‘needle-in-a-haystack’ scenario typical of professional security audits. Our analysis focuses on the relative improvements and structural consistency of the learned embeddings, providing a more realistic assessment of how Deep Learning (DL) models generalize to complex, low-frequency software vulnerabilities. The contributions of this work are:

- An extended graph extraction pipeline capable of generating CFGs, ASTs, and PDGs from C functions of the Linux kernel dataset;
- A set of new graph-based features defined and integrated for AST and PDG representations;
- A modular neural network framework based on DGCNN,

supporting multiple classifier heads and an autoencoder module for embedding pretraining;

- An embedding extraction and external classifier branch that enables DGCNN embeddings to be projected with Principal Component Analysis (PCA) for visualization and evaluated with conventional classifiers such as SVM and RF.

This paper is structured as follows: Background concepts are presented in Section II. Section III presents the methodology followed in this work. Section IV presents the dataset used in this study. Section V presents the results and discussion. Section VI presents the related work and discusses the main challenges of imbalanced datasets for vulnerability detection. The threats to validity are presented in Section VII. Finally, Section VIII concludes this paper and presents the future work.

II. BACKGROUND

This section provides the background concepts on ML models for vulnerability detection, and graph representations extracted from the source code used in this work.

A. Machine Learning for Vulnerability Detection

ML techniques have been increasingly applied to detect software vulnerabilities by learning discriminative patterns directly from code data [4], [13]. Traditional algorithms such as SVMs, DTs, and RFs rely on hand-crafted features like code metrics, static-analysis outputs, or token sequences to classify code units [14], [15], [16]. These methods can generalize across settings but are limited by feature engineering effort and cross-project adaptability [4].

DL reduces dependence on manual features by learning hierarchical representations from raw or minimally processed code [17], [18]. CNNs have been applied to token or sequence views of code to capture local patterns linked to vulnerabilities [17]. However, CNNs operate on regular grids and are less suited to the irregular, hierarchical structure of code. GNNs extend convolution to non-Euclidean domains, modeling program entities and relations as nodes and edges and thus capturing syntactic and semantic dependencies critical for complex vulnerability detection [19], [20].

DGCNN extends graph convolution architectures to graph-level classification by stacking graph convolution layers, applying a *SortPooling* step to order node embeddings, and using fixed-size 1D convolutions for downstream processing [21]. This design enables learning from variable-size graphs while preserving structural cues. DGCNN has performed well in tasks where relational structure matters, such as malware classification from CFGs [7], and are therefore attractive for source-code vulnerability analysis [8].

B. Graph Representations and Feature Extraction

Motivated by the suitability of graph-based DL models for vulnerability detection, we now describe the graph representations and node features used in this work. Several structures can be extracted from the source code. The ones we are using in this work are the following:

- **CFG**: models execution paths between basic blocks, capturing branching and control dependencies, widely used in vulnerability analysis [22], [23].
- **AST**: encodes syntactic structure according to the language grammar and organizes statements hierarchically; often forms the basis for higher-level representations [18].
- **PDG**: integrates control and data dependencies to represent predicate influence and value flow, offering a unified semantic view [24].

These graphs provide structural input to the DGCNN-based framework, enabling learning from syntax, semantics, and dependencies rather than purely lexical tokens. Feature processing is central when preparing graph data for learning [4]. Each node is represented by a feature vector encoding syntactic or semantic properties, such as operation category or memory-management behavior in C functions [8]. The adjacency matrix together with node features feed the DGCNN encoder, which learns embeddings that capture local patterns and global dependencies [21]. Those embeddings are used for downstream classification and visualization, as described in the next section.

III. METHODOLOGY

This section describes the methodology and summarizes both the baseline and the extended pipelines. Fig. 1 illustrates the proposed approach followed in this work. The pipeline separates the workflow into two main components: (i) a Graph Extraction Pipeline that produces adjacency and feature matrices for CFGs, ASTs, and PDGs, and (ii) a Classification Pipeline that consumes those matrices and supports multiple classifiers and an optional Autoencoder (AE) for self-supervised embedding refinement.

We provide reproducibility artifacts in two repositories: (i) graph extraction scripts for each code unit (<https://github.com/joseadp/dgcnn-graph-extraction/>) and (ii) the DGCNN-based classification pipeline (<https://github.com/joseadp/dgcnn-classification/>), including model training, experiment configuration, and evaluation scripts needed to reproduce the reported results.

Different from [8], our approach preserves full, unreduced graphs to retain structural richness and supports both end-to-end and embedding-based evaluation. The following sections detail the pipeline, and the metrics used to evaluate our results.

A. Graph Extraction Pipeline

The extraction pipeline automates checkout of commits, parsing with Joern [9], export of per-function graphs (.dot extension files), and conversion into adjacency and feature matrices. A CSV metadata file drives the process by listing file paths and commit hashes, and a function-to-graph mapping step links each labeled function to its exported .dot file. All successful exports are kept in standardized directories indexed by graph type and commit hash.

We extract three graph types using Joern [9]: (i) CFGs, capturing execution paths and control dependencies, (ii) ASTs,

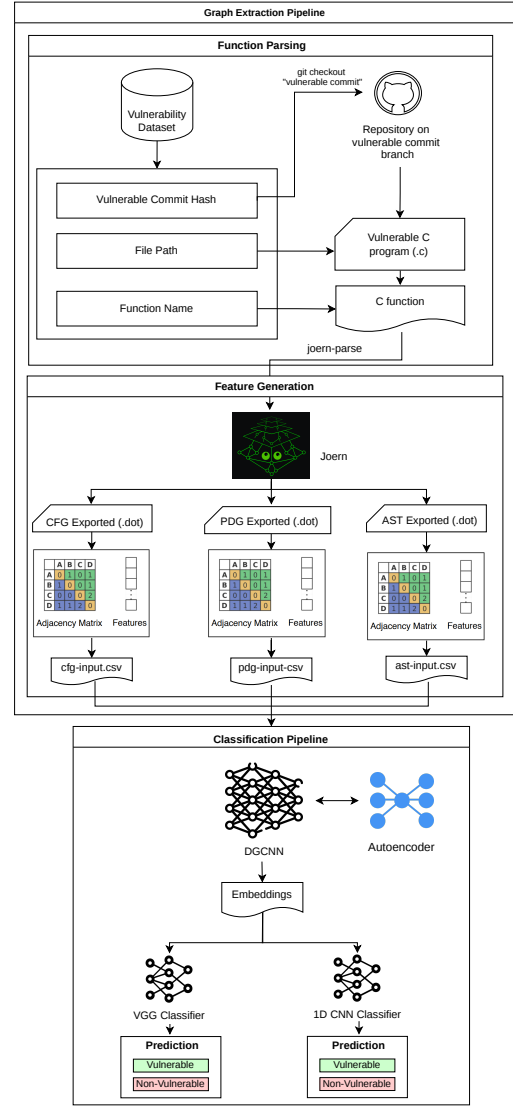


Fig. 1. Extended pipeline including both the graph extraction and neural classification components.

encoding syntactic structure, and (iii) PDGs, integrating control and data dependencies for a semantically rich view. Graphs that fail to parse are skipped and logged.

From each .dot file, we generate two artifacts: (1) a copy of the original .dot graph, and (2) a .txt file with node labels/statements extracted from dot attributes (Fig. 2). A dot parser is used to build a directed graph, produce an adjacency matrix, and extract node-level attributes. The

TABLE I
A SUMMARY OF PROJECTS USED IN OUR WORK (LINES OF CODE OBTAINED FROM OPENHUB [25] ON APRIL 2026)

Projects	Language	Version Control	Lines of Code	Website
Linux Kernel	C	Git	37.8 million	Kernel.org
Mozilla	C++	Git	32.7 million	Mozilla.org
Xen Hypervisor	C	Git	0.669 million	XenProject.org
Apache HTTP Server	C	Git and SVN	1.63 million	Apache.org
GNU C Library (Glibc)	C	Git	1.39 million	GNU.org

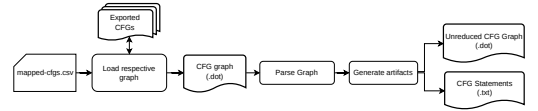


Fig. 2. Example of CFG artifact generation for one function

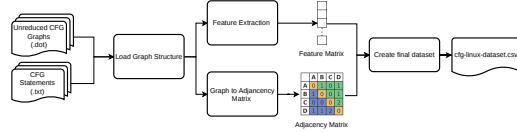


Fig. 3. Example of the final dataset generation step for CFG graphs

adjacency matrix, feature matrix, node count, and binary label are serialized into rows of a Comma-Separated Value (CSV) dataset for efficient loading and batching during training. The overall process of the dataset generation is depicted in Fig. 3

The **Node features** are fixed-length numeric vectors computed per graph type and encoded together with the adjacency matrix. For CFGs, features include vertex-structure metrics (in-degree, out-degree, number of instructions), code-sequence indicators (arithmetic, comparisons, data movement, returns/calls, declarations), and memory-management counts (allocations, deallocations, pointer ops, unsafe functions such as `strcpy` or `scanf`), such as defined in [8]. For ASTs, features mark literals and identifiers, call/block/control constructs, unknown node types, leaf status, and out-degree. Finally, for the PDGs, features capture Data Dependence Graph (DDG) and Control Dependence Graph (CDG) in/out degrees, a control-dependent flag, an is-parameter flag, and total edge counts. The complete list of features can be seen on Table II. All features are normalized and flattened per graph for storage and later use in the DGCNN encoder. The generated mapping of each graph with their functions is presented in Fig. 4.

B. Classification pipeline

The classification pipeline is modular and contains an encoder, pooling module(s), classifier branches, optional autoencoder, and utilities for embedding export and external classifier evaluation. This modularization separates DGCNN encoding from classifier logic and simplifies experiments that swap classifiers or freeze encoder weights after pretraining.

The pipeline first applies min-max normalization to all node features using statistics computed exclusively from the training set to prevent data leakage; this step precedes the DGCNN encoder to stabilize gradient flow in the scale-sensitive graph convolution layers. Following normalization, the encoder uses four stacked GraphConv layers with ReLU

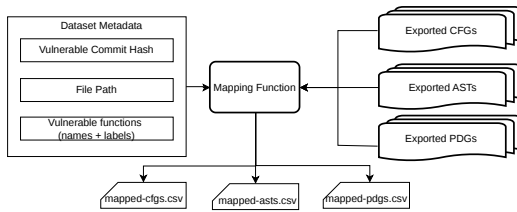


Fig. 4. Function-to-graph mapping for all graphs

TABLE II
NODE FEATURES EXTRACTED FOR EACH GRAPH REPRESENTATION.

Graph	Group (# feat.)	Features
CFG	Vertex (3)	In-degree, Out-degree, #Statements
	Code (8)	Numeric, Transfer, Call, Arithmetic, Comparison, Move, Termination, Declaration
	Memory (8)	Allocation, Deallocation, Pointer Assignment, Address-of, Convert Unsafe, String Unsafe, Scanf Unsafe, Other Unsafe
AST	Node Type (6)	Literal, Identifier, Call, Block, Control Structure, Unknown
	Structure (2)	Leaf, Out-degree
PDG	Dependency (5)	DDG In-degree, DDG Out-degree, CDG In-degree, CDG Out-degree, Total Edges
	Semantic (2)	Control Dependent, Parameter

activations to produce node-level embeddings, which are aggregated into graph-level representations via SortPooling for the Conv1D classifier or Adaptive Max Pooling (AMP) for the VGG adapter. When AE pretraining is enabled, it runs after normalization using the same encoder, which is subsequently frozen before classifier optimization. These steps are detailed in the following subsections.

1) *DGCNN Encoder and Pooling*: The encoder stacks graph-convolution layers interleaved with ReLU activations and optional dropout to produce node embeddings. Two pooling strategies are supported: SortPooling [21], which orders node embeddings and selects the top k nodes to form a fixed-size sequence, and AMP, which produces a fixed-size tensor by adaptively selecting maxima across regions. SortPooling feeds 1D convolutions and preserves ordered structural cues, while AMP produces an image-like tensor suitable for deep 2D CNNs such as VGG.

2) *Normalization*: All node features are normalized before training using min-max scaling based on the training set range, with safeguards for constant-value features to avoid division by zero. This strategy was chosen for stability and compatibility with the non-negative features derived from code graphs.

3) *Classifier Architectures*: Two main classifier branches are implemented and selectable by a configuration flag. The first is a lightweight 1D convolutional classifier inspired by DGCNN, consisting of two Conv1D layers (kernel size 5), a flattening step, a 64-unit dense layer, dropout, and a two-unit output layer. This branch is computationally efficient and pairs naturally with SortPooling.

The second branch adapts embeddings into a pseudo-image and uses a VGG11-inspired backbone. The adapter applies a 13×13 2D convolution and AMP to produce tensors of shape $(B, C, 30, F)$, where B is the batch size, C is the number of convolutional channels, and F corresponds to the original embedding dimensionality. The output is consumed by five convolutional blocks and two 4096-unit fully connected layers. This heavier pathway aims for higher precision when sufficient data and stability are available. Both classifiers can be trained end-to-end or on frozen encoder embeddings.

4) *Embedding Extraction and External Classifiers*: The pipeline can export graph-level embeddings, logits, and predictions for downstream analysis. Conv1D exports are taken after SortPooling, while VGG exports perform global pooling

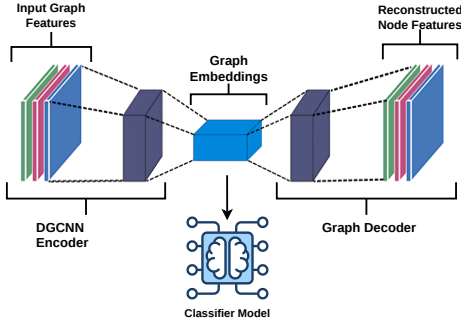


Fig. 5. Overview of the Autoencoder (AE) extension (adapted from [26])

over the spatial map to yield a compact vector. An auxiliary script supports PCA (default 20 components) for visualization (two 2-D plots: colored by ground-truth and by prediction outcome) and trains conventional classifiers (RF with 200 trees and SVM) using balanced sample weights to account for class imbalance. The script saves classification reports and confusion matrices for reproducibility.

5) *Autoencoder (AE) Extension*: An optional AE reconstructs the original node feature matrix from graph-level embeddings to encourage topology-preserving representations. The autoencoder uses the DGCNN Encoder as encoder and a decoder that outputs a reconstructed matrix of shape $[B, N, F]$, where B is the batch size (number of graphs per mini-batch), N is the number of nodes per graph (padded if needed) and F is the number of features per node. Graphs are zero-padded or truncated to a target node count set at the 95th percentile of graph sizes to balance coverage and memory usage. Training minimizes Mean Squared Error (MSE) between original and reconstructed node features, and the encoder may be frozen after pretraining before classifier optimization. This process is depicted in Fig. 5.

The AE regularizes the encoder by forcing retention of reconstructible node-level information and thereby can improve downstream classification robustness.

C. Training Setup and Evaluation Metrics

All models are trained with the Adam optimizer, minimizing Cross-Entropy Loss with optional class weights to address imbalance. Training logs per-epoch loss and accuracy and saves training curves for inspection. Evaluation uses Accuracy, Precision, Recall, and F1-score, with F1 prioritized due to severe class imbalance where single-metric gains can be misleading. Each function prediction can be classified as:

- **True Positive (TP)**: Vulnerable functions correctly classified as vulnerable;
- **False Positive (FP)**: Non-vulnerable functions incorrectly classified as vulnerable;
- **True Negative (TN)**: Non-vulnerable functions correctly classified as non-vulnerable;
- **False Negative (FN)**: Vulnerable functions incorrectly classified as non-vulnerable.

From these, the following evaluation metrics are computed:

- **Accuracy**: $\text{Accuracy} = \frac{TP+TN}{TP+FP+TN+FN}$, the proportion of correctly classified functions over the total.

- **Precision**: $\text{Precision} = \frac{TP}{TP+FP}$, the proportion of predicted vulnerable functions that are truly vulnerable.
- **Recall**: $\text{Recall} = \frac{TP}{TP+FN}$, the proportion of actual vulnerable functions that are correctly identified.
- **F1-score**: $F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$, the harmonic mean of precision and recall, providing a single metric that balances both.

While all four metrics are reported, the F1-score is the most relevant in this domain. Given the severe class imbalance of real-world datasets, high precision alone can be misleading (e.g., a classifier collapsing into predicting a single class). These metrics are calculated on the test set after training, and in some experiments, also on PCA-reduced embeddings using external classifiers such as RF and SVM (see Section III-B4).

IV. DATASET

This section describes the dataset used throughout this work. The dataset [12] aggregates vulnerable and non-vulnerable C/C++ code units (e.g., files, functions) from five prominent open-source projects: Linux Kernel, Mozilla, Xen Hypervisor, Apache HTTP Server, and the GNU C Library (glibc). A summary of the projects can be found on Table III. This work focuses exclusively on C functions extracted from the Linux Kernel project. This choice reflects the large volume of vulnerabilities in this repository, its relevance as a mature and low-level system software, and the fact that most vulnerability types in the dataset, particularly memory management issues, are highly prevalent in the Linux Kernel.

This work starts from the study by *Pereira et al.* [8], which relied solely on Control Flow Graphs (CFGs). Here, the same dataset is reused to also extract Abstract Syntax Trees (ASTs) and Program Dependence Graphs (PDGs), extracted using Joern [9]. These additional representations were not included in the previous work and allow a broader evaluation of graph-based vulnerability detection.

The dataset is used as input for graph extraction and for feature design and extraction across different graph types throughout this work. The remainder of this section describes the structure and labeling process of the dataset, outlines the main vulnerability categories and their Common Weakness Enumeration (CWE) mappings, and details the size and sampling strategy applied to the Linux subset used in this work.

A. Structure and Labels

The dataset provides three types of code units: files, functions, and classes (with `structs` treated as classes) [12]. In this work, the analysis focuses only on functions as they provide a practical level of granularity where vulnerabilities

TABLE III
A SUMMARY OF PROJECTS USED IN OUR WORK (LINES OF CODE OBTAINED FROM OPENHUB [25] ON APRIL 2026)

Projects	Language	Version Control	Lines of Code	Website
Linux Kernel	C	Git	37.8 million	Kernel.org
Mozilla	C++	Git	32.7 million	Mozilla.org
Xen Hypervisor	C	Git	0.669 million	XenProject.org
Apache HTTP Server	C	Git and SVN	1.63 million	Apache.org
GNU C Library (Glibc)	C	Git	1.39 million	GNU.org

can often be identified. Functions are smaller and more consistent than entire files or classes, while still containing enough structural and semantic information for effective vulnerability detection. This aligns with the approach defined in [8], which this work builds upon, with functions as the unit of analysis as established previously. Each function entry is then mapped to its source file and labeled as either vulnerable or neutral. A function is labeled vulnerable if it overlaps with a security patch; otherwise, it is labeled neutral.

Each entry includes detailed metadata, such as: (i) File path, function name, commit hashes, author dates, and changed-line intervals; (ii) Topological index, parent count, and tag names from version control history; (iii) Function metadata (signature, line ranges, and vulnerability label); (iv) CVE IDs (when available); (v) Last-change commit hashes.

In addition to versioning and labeling information, each function entry is enriched with vulnerability-relevant data:

- **Software Metric (SM):** extracted using the Understand tool, including metrics such as Lines of Code (LOC), cyclomatic complexity, fan-in/out, and coupling measures;
- **SAT Alerts:** generated by two SATs (*i.e.*, Flawfinder [27] and Cppcheck [28]), with alerts mapped to functions based on line ranges.

This structure enables downstream tasks such as graph generation, feature extraction, and training ML models for vulnerability classification.

B. Types of Vulnerabilities

The dataset includes 13,752 known vulnerabilities of five major C/C++ projects, distributed across several CWE identifiers. These CWEs were grouped into categories inspired by OWASP secure development practices, proposed in the original dataset [12], [29].

The updated dataset version presented in [29] expands the set of vulnerabilities from 5,214 to 13,752 and revises the treatment of CWE identifiers. Several identifiers used in the earlier classification are now marked as deprecated, prohibited, or discouraged by the CWE project [30], such as: **CWE-264: Permissions, Privileges, and Access Controls** [31], **CWE-399: Resource Management Errors** [32], **CWE-17: Code** [33], and **CWE-20: Improper Input Validation** [34].

Although this work uses the earlier dataset version [12], the CWE-to-category mapping was revised to reflect updates from the most recent version [29] and the current CWE project [30]. Table IV summarizes the categorization, showing updated percentages for the most frequent categories and counts for the remainder.

C. Linux Dataset Size and Sampling Strategy

The Linux subset used in this work contains around 100,000 C functions, with the vulnerable class at roughly 2.6% across CFG, PDG, and AST (Table V). These values are consistent across graph types and illustrate the severe class imbalance that characterizes the dataset. Minor differences in the number of instances across graph types arise from the extraction process, since parsing is performed at the file level and

TABLE IV
DATASET CWE CATEGORIZATION (BASED ON [29]).

Category	CWEs	Vuln. Count (%)
Memory Management	119, 362, 416, 476, 824	3392 (24.66%)
Input Validation	78, 79, 91, 94, 134, 189	976 (7.10%)
Permission	255, 269, 284, 287, 352	— (—)
Data Protection	199, 200	— (—)
Coding Practices	19, 254	— (—)
Cryptography	310	— (—)
System Configuration	16	— (—)
File Management	22, 59	— (—)
Output Encoding	—	— (—)
Error Handling and Logging	—	— (—)
Communication Security	—	— (—)
Database Security	—	— (—)
Unmapped CWEs	—	- (—)
Total	—	13,752

Counts for Memory Management and Input Validation categories from the dataset [29]. For the remaining categories, absolute counts were not published in the newer version, and only the total of 13,752 vulnerabilities is known. All deprecated, prohibited, or discouraged CWEs were removed from the mapping.

TABLE V
GRAPH STATISTICS FOR THE FULL LINUX DATASET
(PRE-UNDERSAMPLING)

Graph Type	Total Graphs	Vulnerable (%)	Avg. Nodes
CFG (full)	101,193	2,643 (2.61%)	41.54
PDG (full)	102,209	2,674 (2.62%)	41.05
AST (full)	98,922	2,547 (2.57%)	87.02

certain representations, particularly ASTs, are more sensitive to incomplete contextual information, resulting in a lower number of successfully generated graphs. For example, when a file depends on a struct defined in another file, Joern may fail to correctly extract its AST. Consequently, such graphs are not successfully generated and are therefore excluded from the final dataset, leading to differences in the number of extracted graphs across graph types.

Running all experiments on nearly 100K unreduced functions per graph type, with several seeds and classifier variants, would have taken too long to be practical. In contrast to some related works that apply preprocessing (*e.g.*, slicing or filtering out large functions) to shrink graph size, this study deliberately kept graphs unreduced to better assess model performance and robustness. To keep the experiments feasible, a fixed-size stratified sample of 20,000 functions was drawn for each graph type. Stratification preserved the original class imbalance, while the reduced size allowed training and evaluation to be carried out consistently across graph types (Table VI).

This undersampling strategy provides a good compromise: the reduced datasets remain representative of the Linux project natural imbalance while making it possible to run multiple seeds and configurations within the available compute resources. The next section presents the results obtained when applying the methodology to the dataset presented in this

TABLE VI
GRAPH STATISTICS FOR THE SAMPLED LINUX DATASETS (20,000
FUNCTIONS EACH)

Graph Type	Vulnerable (%)	Vulnerable Count	Avg. Nodes
CFG (undersampled)	2.61%	522	41.35
PDG (undersampled)	2.61%	523	41.26
AST (undersampled)	2.57%	514	88.46

TABLE VII
AVERAGE F1-SCORES (VULNERABLE CLASS) ACROSS REPRESENTATIONS
AND CONFIGURATIONS

Test ID	Graph Type	Classifier	Autoencoder	Mean F1 $\pm$ Std
T1	CFG	Conv1D	No	0.0861 $\pm$ 0.0061 $\uparrow$
T2	CFG	VGG	No	0.0955 $\pm$ 0.0321 $\uparrow$
T3	CFG	Conv1D	Yes	0.1013 $\pm$ 0.0057 $\uparrow$
T4	CFG	VGG	Yes	0.0656 $\pm$ 0.0225 $\uparrow$
T5	AST	Conv1D	No	0.0869 $\pm$ 0.0046 $\uparrow$
T6	AST	VGG	No	0.0545 $\pm$ 0.0177 $\downarrow$
T7	AST	Conv1D	Yes	0.0500 $\pm$ 0.0000 $\downarrow$
T8	AST	VGG	Yes	0.0472 $\pm$ 0.0306 $\downarrow$
T9	PDG	Conv1D	No	0.0866 $\pm$ 0.0227 $\uparrow$
T10	PDG	VGG	No	0.0816 $\pm$ 0.0087 $\uparrow$
T11	PDG	Conv1D	Yes	0.0841 $\pm$ 0.0188 $\uparrow$
T12	PDG	VGG	Yes	0.0634 $\pm$ 0.0495 $\downarrow$
Baseline [8]	CFG	VGG	No	0.0656 $\pm$ 0.0006

Note: $\uparrow$ and $\downarrow$ denote increase or decrease relative to the baseline [8]. The best configuration is represented in **bold**.

section, followed by their discussion.

V. RESULTS AND DISCUSSION

This section presents and interprets the experimental results of the proposed vulnerability detection framework. Each experiment is accompanied by a brief discussion of its implications, highlighting the strengths and limitations of different representations, architectures, and learning strategies.

A. Experimental Setup

All experiments were conducted on a multi-GPU lab server equipped with 3 $\times$ GTX 1080 Ti and 1 $\times$ TITAN V (12GB) GPUs, using CUDA 12.8, PyTorch 2.3.0, and DGL 2.2.1. Training was repeated over five random seeds per configuration to measure stability, using stratified splits and weighted random sampling to mitigate class imbalance. The same default hyperparameters were applied across tests, including a hidden dimension of 32, dropout of 0.3, batch size of 10, and 30 training epochs. Each experiment combined one of three graph types (CFG, PDG, AST) with two classifiers (Conv1D, VGG), both with and without an AE pretraining stage.

B. Overall Performance Trends

The primary metric for comparison was the F1-score for the vulnerable class. It is important to note that the high majority-class bias (97.4% neutral) naturally suppresses absolute F1-score values compared to studies using balanced data. However, the Conv1D+AE configuration (T3) achieved the highest and most consistent results, indicating that the model is successfully identifying discriminative patterns despite the extreme scarcity of training examples. This stability is a key requirement for practical tools intended to reduce developer burden without collapsing into trivial single-class predictions. The results discussed in this section are presented in Table VII, as well as the baseline results used in the evaluation.

1) *CFG-Based Results:* The four CFG configurations (T1–T4) revealed consistent patterns. Conv1D-based models were the most stable and achieved the best trade-off between recall and precision. In contrast, VGG-based classifiers were more volatile, often favoring one class excessively.

Without Autoencoder (AE) pretraining (T1, T2), Conv1D yielded a mean F1-score of 0.0861, while VGG reached 0.0955 but with far higher variance across seeds. Introducing AE pretraining (T3, T4) amplified this difference: the Conv1D+AE setup (T3) achieved the highest mean F1-score (0.1013 $\pm$ 0.0057) and showed the most consistent results across runs, while VGG+AE (T4) collapsed in several runs, producing trivial predictions. These findings directly address **RQ1** and **RQ3**, showing that autoencoder pretraining improves both embedding quality and classifier stability, but only when paired with a lightweight Conv1D head.

2) *AST-Based Results:* The AST experiments (T5–T8) exposed stronger instability, especially when paired with VGG or Autoencoder pretraining. The baseline Conv1D setup (T5) achieved an average F1 of 0.0869, comparable to the CFG baseline, but both AE-enabled (T7, T8) and VGG-based variants (T6, T8) exhibited frequent mode collapse, predicting either all functions as vulnerable or none. The best-performing AST configuration, T5, therefore did not surpass the CFG results. This answers **RQ2** negatively: ASTs, though richer in syntactic structure, did not provide measurable gains in vulnerability detection within this feature and model setup.

3) *PDG-Based Results:* PDG configurations (T9–T12) also displayed high variance across seeds. Conv1D-based variants achieved moderate and consistent recall values (mean F1 $\approx$ 0.0866 for T9 and 0.0841 for T11), while VGG-based runs (T10, T12) showed high accuracy but low recall, indicating a bias toward the majority class. Autoencoder pretraining did not yield consistent improvements and occasionally increased instability. Although PDGs encode deeper semantic and dependency relations, they did not outperform CFGs in isolation. These outcomes provide further evidence for **RQ2** and **RQ3**, indicating that richer graph semantics alone are insufficient without tailored feature sets and larger training data.

Table VII summarizes the main outcomes across all experiments. The key findings can be summarized as follows:

- The combination of full CFGs, Conv1D classifier, and AE pretraining achieved the highest and most stable F1-scores, surpassing prior CFG-based baseline [8];
- Classifier complexity was inversely correlated with stability: VGG networks often overfit or collapsed entirely, while Conv1D produced steady behavior (**RQ4**);
- Autoencoder pretraining enhanced embedding consistency in Conv1D configurations but destabilized VGG runs, highlighting the interaction between pretraining and model capacity;
- Neither AST nor PDG representations outperformed CFGs, suggesting that their added syntactic and semantic depth does not directly translate into better discriminative power under the current feature design.

4) *Degradation in Deeper Configurations:* The results in Table VII reveal consistent degradation in configurations T4, T7, T8, and T12 due to VGG-based classifiers, AE pretraining, or both. Because of a severe 97.4% majority-class imbalance ($\sim$ 520 vulnerable training examples), the VGG11 backbone’s two 4096-unit fully connected layers are over-parameterized,

TABLE VIII
COMPARISON OF T9 AND T11 FOR IDENTICAL SPLIT AND SEED (75).

Configuration	Accuracy	Precision	Recall	F1
T9 Run 4 (no AE)	0.5677	0.0448	0.7643	0.0847
T11 Run 4 (with AE)	0.6300	0.0494	0.7197	0.0924

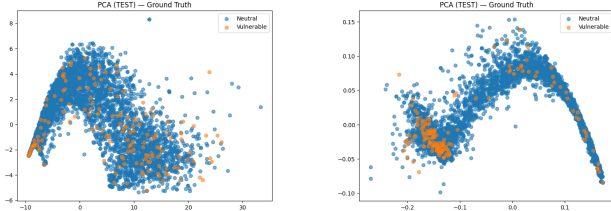


Fig. 6. PCA projection of test embeddings colored by ground-truth labels (neutral vs. vulnerable). Left: no AE. Right: with AE.

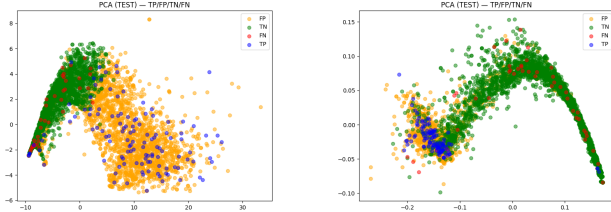


Fig. 7. PCA projection of test embeddings coloured by prediction outcome (TP, FP, TN, FN). Left: no AE. Right: with AE.

causing models to collapse to trivial majority-class predictions across seeds; this is reflected in low mean F1-scores and high standard deviations (T8: 0.0472 ± 0.0306 ; T12: 0.0634 ± 0.0495). AE pretraining compounds this instability; while it improves Conv1D performance on CFGs (T1 $\rightarrow$ T3: $0.0861 \rightarrow 0.1013$), it degrades VGG on the same representation (T2 $\rightarrow$ T4: $0.0955 \rightarrow 0.0656$) because optimizing for reconstruction fidelity produces embeddings an over-capacity classifier cannot exploit. This degradation is most severe on ASTs, where T7 collapses entirely (0.0500 ± 0.0000) compared to the AST baseline T5 (0.0869); because AST graphs are larger (~ 88 average nodes) but informationally sparser (8 node features versus 19 for CFG), pretraining reinforces noise rather than the minority-class vulnerability signal.

C. Embedding Analysis – PDG With and Without Autoencoder

This section provides a focused analysis of the DGCNN embeddings for PDGs, comparing configurations with and without AE pretraining. Due to space constraints, we analyze two representative runs with identical splits and seed: T9 Run 4 (PDG+Conv1D, no AE) and T11 Run 4 (PDG+Conv1D, with AE). As shown in Table VIII, the autoencoder yielded a measurable gain in vulnerable-class F1 ($0.0847 \rightarrow 0.0924$), while preserving overall performance.

PCA set-up. The encoder’s node-level outputs were reduced to 20 dimensions using PCA, and the first two components were visualized. Projections are shown both by ground-truth label and by prediction outcome (TP/FP/TN/FN), such as depicted in Figs. 6 and 7.

TABLE IX
PERFORMANCE OF RF AND SVM ON ENCODER EMBEDDINGS (VULNERABLE CLASS).

Configuration	Classifier	Acc.	Prec.	Rec.	F1
PDG No-AE	RF	0.9648	0.0645	0.0255	0.0365
PDG No-AE	SVM	0.6793	0.0540	0.6815	0.1001
PDG + AE	RF	0.9708	0.0500	0.0064	0.0113
PDG + AE	SVM	0.6660	0.0536	0.7070	0.0997

Key observations. The PCA projects reveal that even when the gains in the F1-score are modest, the AE pretraining (T11) forces vulnerable samples into a more dense and coherent cluster compared to the baseline without AE (T9). This structural improvement in the latent space suggests that the DGCNN is capturing the underlying semantics of vulnerabilities, even if the final classifier boundary remains conservative due to class imbalance. This demonstrates that the bottleneck lies in the feature granularity and signal-to-noise ratio rather than the model’s capacity to learn.

Classical models on embeddings. To evaluate whether the encoder captures discriminative patterns, RF and SVM classifiers were trained directly on the embeddings. RF achieved high accuracy but low recall for the minority class, while SVM traded accuracy for significantly higher recall, yielding slightly better F1 in both runs, evidence that the embeddings encode exploitable signal. The results can be seen in Table IX.

Takeaway. Autoencoder pretraining improves the organization of the embedding space (clearer vulnerable grouping) and modestly raises F1. Combined with slicing and refined PDG features, this structure is expected to yield stronger downstream performance.

D. Discussion

The experiments revealed clear patterns of stability, variability, and collapse across graph representations, classifier architectures, and the use of autoencoder pretraining. Some configurations maintained stable recall and F1-scores, whereas others showed severe seed sensitivity or degenerated into predicting a single class. These outcomes expose the delicate balance between graph complexity, classifier capacity, and representation quality.

The Conv1D classifier consistently produced more stable results than VGG, particularly in CFGs, where it achieved low variance and balanced precision–recall values across seeds. In contrast, the deeper VGG classifier exhibited high sensitivity to training noise, often collapsing into single-class predictions or showing strong bias toward the non-vulnerable class. This confirms **RQ4**, demonstrating that classifier complexity strongly influences both stability and performance: lightweight architectures are more robust under limited and imbalanced data conditions.

Autoencoder pretraining showed a mixed but informative effect. For Conv1D-based configurations on CFGs, pretraining improved the balance between recall and precision, leading to the highest and most stable F1-scores observed in this study. In contrast, on ASTs and PDGs, the benefits were inconsistent or negligible, and VGG models generally deteriorated with pre-training. Nevertheless, the embedding analysis in Section V-C

showed that pretraining led to a denser and more structured grouping of vulnerable functions, indicating improved latent space organization. These observations address **RQ3**, showing that AE pretraining enhances representation quality even when its impact on classification metrics is limited. With refined features and stronger class balancing, such structured embeddings could yield greater improvements in downstream detection.

Graph complexity also had a significant influence on model behavior. ASTs, being large and syntactically detailed, amplified instability and overfitting in both classifiers, suggesting that full-scale trees are difficult to process without prior slicing or abstraction. CFGs, being compact and associated with well-engineered node features, produced the most consistent and interpretable results. PDGs fell between these extremes, capturing both control and data dependencies but occasionally suffering from noise and imbalance. These findings indicate that richer structures do not automatically translate into better detection performance; feature quality and input granularity play a far more decisive role. Compared with baseline results [8] (Table VII), we could almost double the value of F1-score. This answers **RQ2** by showing that, although AST and PDGs offer deeper semantics, CFGs remain the most practical and effective representation under the current feature design.

Class imbalance emerged as a persistent limitation across all experiments. Several configurations collapsed into predicting a single dominant class, highlighting the need for more robust balancing strategies. This behavior aligns with the trends identified in the literature review, where high-performing models typically rely on more balanced datasets. The results reaffirm that imbalance in large real-world codebases remains the key bottleneck for machine learning-based vulnerability detection.

Among all metrics, the F1-score provided the most meaningful measure of progress, as accuracy alone often concealed severe bias. By this criterion, the most reliable configuration was the Conv1D classifier on full CFGs with autoencoder pretraining, which achieved the best compromise between recall and precision while maintaining stability across runs. This configuration can serve as a strong baseline for future research on graph-based vulnerability detection.

Overall, the results suggest that model depth and graph complexity are secondary to the quality and granularity of the graph representations themselves. Future improvements should focus on refined feature engineering, program slicing to reduce graph noise, and class balancing techniques tailored to each classifier’s sensitivity. Lightweight architectures such as Conv1D, combined with improved input representations, appear to offer the most promising path forward for practical and interpretable vulnerability detection frameworks.

E. Practical Implications

These results offer concrete lessons for practitioners applying DGCNNs to vulnerability detection under realistic class imbalance. First, lightweight classifiers should be preferred over over-parameterized architectures like VGG, which collapse into majority-class predictions under a 97.4% imbalance where Conv1D remains stable. Second, node feature design

matters more than structural complexity, as CFGs consistently outperformed the richer but sparser ASTs and PDGs. Third, AE pretraining is beneficial but not universal; it improves Conv1D on CFGs (T1 $\rightarrow$ T3: 0.0861 $\rightarrow$ 0.1013) but degrades VGG and noisier representations, meaning embedding discriminability must be verified before freezing encoders. Finally, because accuracy is misleading at this imbalance level, F1-score and recall must serve as primary metrics, with confusion matrices reported explicitly to expose FN rates.

VI. RELATED WORK

Xuan et al. [35] propose a detection method based on code property graphs that integrates ASTs, CFGs, and PDGs. They evaluate on FFmpeg and QEMU and report high metrics (recall 97%, F1 93%, accuracy 96%), but their experiments rely on datasets with unusually high vulnerable ratios (*e.g.*, SARD), and they do not describe explicit imbalance mitigation.

Wang et al. [36] propose DeepVulSeeker, which combines multiple graph views with MetaPath and pre-trained models (*e.g.*, UniXcoder) and adds a Graph Representation Self-Attention module. They evaluate across six datasets and report strong results on CWE subsets and modest F1-score on the projects FFmpeg and QEMU, but do not detail imbalance-handling strategies.

Zhou et al. [37] introduce Devign, which integrates ASTs, CFGs, Data Flow Graphs (DFGs), and token sequences, initializes node features with Word2Vec, and uses gated graph networks plus a convolutional readout. Evaluations on 4 projects show meaningful F1-score improvements (up to 8.68% improvement) over baselines, using manually labeled datasets but not describing imbalance mitigation.

Cao et al. [38] present MVD+, which targets memory-related flaws with ASTs and System Dependence Graphs (SDGs), uses program slicing to reduce noise, and trains flow-sensitive GNNs with BPE/ASTNN embeddings; it reports significant gains on MemoryVul [39], a dataset with a more favorable vulnerable ratio than many real-world corpora. The MemoryVul dataset comprises both synthetic and real-world examples, and MVD+ reports performance improvements of up to 28.02% in recall and 22.46% in precision over existing baselines. Although the dataset is not balanced, with about one-fifth of the samples labeled as vulnerable, this ratio can still be considered more favorable than the highly skewed distributions typically found in large codebases.

Hin et al. [40] introduce LineVD, which applies PDGs/CDGs and CodeBERT embeddings with a Graph Attention Network (GAT) backbone on Big-Vul, and explicitly incorporates function-level context. The framework demonstrates substantial performance improvements, reporting an 104% increase in F1-score when compared to other state-of-the-art approaches. Additionally, LineVD is one of the few works that operates on a dataset with a vulnerable ratio that remains closer to real-world conditions. As a result, the reported F1-scores are lower than those achieved by studies relying on SARD or heavily balanced subsets, which highlights how dataset composition has a decisive impact on the metrics.

Zheng et al. [41] introduce VulSPG, a vulnerability detection framework that uses Slice Property Graphs (SPGs) that combine control, data, and call dependencies, enriches node representations with multi-level attention, and uses R-GCNs; it reports robust function- and slice-level F1-scores on FFmpeg and QEMU. Experimental results demonstrate that SPGs consistently outperform alternative graph representations: at function-level, VulSPG achieves F1-scores of 64.7% on FFmpeg and 58.6% on QEMU; at slice-level, the model reaches an F1-score of 94.3% and accuracy of 97.1%. Additionally, the function-level analysis shows improved performance, with F1-scores of 69.8% for FFmpeg and 61.2% for QEMU, underscoring the method robustness across granularities.

Wu et al. [42] introduce VulCNN, which converts PDGs into image-like representations using centrality measures and sent2vec, and classifies them with CNNs. Evaluated on a dataset comprising 40,657 functions, VulCNN achieves a True Positive Rate (TPR) exceeding 90%, a True Negative Rate (TNR) above 80%, and an overall accuracy of approximately 85%. This dataset comprised roughly one-third vulnerable functions, which, while not fully balanced, may have contributed to the relatively high reported performance when compared to more unbalanced real-world scenarios.

Wang et al. [43] propose DC-GAR, a graph-based vulnerability detection approach that combines structural graph properties with random walk-based features in a dual-channel graph neural network. The method operates on PDGs and processes complementary representations through parallel GCN-based approaches. Evaluated on a dataset of 29,514 functions, DC-GAR achieves an F1-score of approximately 86.86% and outperforms baselines such as Devign [37] and VulCNN [42]. However, the evaluation relies on a relatively balanced dataset with resampling, which limits direct comparability with highly imbalanced real-world settings.

Li et al. [44] introduce SySeVR, a comprehensive DL framework for detecting software vulnerabilities in C/C++ codebases by integrating both syntactic and semantic program representations. SySeVR embeds the vulnerability candidates with Word2Vec, and uses BGRUs for learning. The evaluation is conducted on a dataset comprising 126 vulnerability types sourced from NVD and SARD, where the BGRU model achieves the best results, with 96% accuracy and an F1-Score of 85.8% across four specific vulnerability categories.

Overall, prior work demonstrates a diversity of graph choices, feature strategies, and model architectures. However, differences in dataset composition, particularly regarding class imbalance, make direct comparisons challenging and can affect conclusions about generalization to real-world scenarios. This practical experience report contributes by systematically comparing CFG, AST, and PDG inputs, contrasting lightweight and deep classifiers, and studying AE pretraining on a highly imbalanced Linux-kernel function dataset.

VII. THREATS TO VALIDITY

This section discusses the limitations that may affect the generalizability, reliability, and reproducibility of the results.

Despite efforts to design a rigorous methodology, threats remain across construct, internal, and external validity.

Construct validity concerns whether the concepts were correctly operationalized. The graph extraction pipeline is tightly coupled to the metadata structure of the dataset from [12], and adapting it to other datasets (e.g., SARD, QEMU, or NVD-derived corpora) would require changes in metadata handling and parsing logic. Additionally, a small number of functions could not be parsed or converted into graphs due to Joern limitations or source code inconsistencies, leading to their exclusion. However, all graphs were generated using the same workflow, ensuring consistent preprocessing. Another concern is the disparity in feature definitions: CFGs relied on more mature features, while PDG and AST used simpler ones, which may have affected performance without implying their unsuitability.

Internal validity refers to if the observed effects arise from the studied factors rather than confounders. Class imbalance remained a challenge. Although sampling-based weighting increased minority representation, instability persisted in some cases (e.g., AST-based and VGG-based experiments). Alternative strategies (e.g., class weighting and focal loss) were explored, but the reported results use the most stable configuration. Another limitation is the restricted hyperparameter tuning, based on manual exploration rather than exhaustive search, meaning better configurations may exist.

External validity concerns how well the findings generalize beyond the study setting. All experiments used only the Linux Kernel subset from [12], which limits generalization to other projects (e.g., Mozilla, or Xen). This choice was driven by time constraints and the focus on modularizing the methodology, although the pipeline supports future extensions. Additionally, we compared our work with baseline results from [8]. This work used only Linux Kernel dataset, and to have consistent results with the baseline, we limited our experiments to Linux Kernel. Consequently, it also limits the generalization of our findings. Another threat is the use of full, unsliced graphs, which, while enabling direct evaluation of raw representations, may impact performance and limit comparability with prior work that uses graph slicing or different granularities. This limitation is discussed in Section V-D.

VIII. CONCLUSION AND FUTURE WORK

Software vulnerabilities remain a main concern, and traditional static and dynamic analyses rely on expert-crafted rules, frequently leading to a high number of FPs and FNs. ML approaches, particularly graph-based models such as DGCNNs, offer an alternative by learning patterns directly from code. This work evaluated DGCNNs for vulnerability detection in C functions using CFGs, ASTs, and PDGs, supported by a reproducible pipeline combining graph extraction, two CNN-based classifiers, and an AE under highly imbalanced conditions.

Experiments revealed clear differences between representations and classifiers, with Conv1D showing greater stability than VGG, particularly on CFGs. Overall, CFGs delivered the

most consistent results, while ASTs and PDGs were limited by simpler features and noise. Although performance remains modest, results demonstrate the feasibility of applying DGCNNs to highly imbalanced real-world data (2.6% vulnerable functions) and highlight the importance of feature design over model complexity. The proposed pipeline provides a reproducible and extensible baseline.

Future work should improve ASTs and PDGs with richer features and graph slicing, extend the pipeline to additional projects for cross-project evaluation, add an LLM-based module to support the classification, and improve sampling and imbalance-handling strategies (e.g., focal loss).

ACKNOWLEDGMENTS

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 - Centre for Informatics and Systems of the University of Coimbra, <https://doi.org/10.54499/UID/00326/2025>.

REFERENCES

- [1] Verizon, “2024 data breach investigations report,” Verizon, Tech. Rep., 2024, Accessed: 2024-12-12. [Online]. Available: <https://www.verizon.com/business/resources/T22a/reports/2024-dbir-data-breach-investigations-report.pdf>.
- [2] N. S. Harzevili et al., *A survey on automated software vulnerability detection using machine learning and deep learning*, 2023. arXiv: 2306.11673 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2306.11673>.
- [3] vFunction, *Static vs. dynamic code analysis: Advantages and use cases*, Accessed: 2024-12-14, 2024. [Online]. Available: <https://vfunction.com/blog/static-vs-dynamic-code-analysis/>.
- [4] Y. Meng, H. Zhuang, Z. Lin, and Y. Jia, “A survey on machine learning-based detection and classification technology of malware,” in *2021 International Conference on Computer Information Science and Artificial Intelligence (CISAI)*, 2021, pp. 783–792. DOI: 10.1109/CISAI54367.2021.00158.
- [5] Y. Hu et al., “Interpreters for GNN-Based Vulnerability Detection: Are We There Yet?” In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023, Seattle, WA, USA: Association for Computing Machinery, 2023, pp. 1407–1419, ISBN: 9798400702211. DOI: 10.1145/3597926.3598145.
- [6] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *arXiv preprint arXiv:1609.02907*, 2016.
- [7] J. Yan, G. Yan, and D. Jin, “Classifying malware represented as control flow graphs using deep graph convolutional neural network,” in *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, Portland, OR, USA, 2019, pp. 52–63. DOI: 10.1109/DSN.2019.00020.
- [8] J. D. Pereira, N. Lourenço, and M. Vieira, “On the use of deep graph cnn to detect vulnerable c functions,” in *Proceedings of the 11th Latin-American Symposium on Dependable Computing*, ser. LADC ’22, Fortaleza/CE, Brazil: Association for Computing Machinery, 2023, pp. 45–50, ISBN: 9781450397377. DOI: 10.1145/3569902.3569913.
- [9] Joern Contributors, *Joern documentation*, Accessed: December 23, 2024, 2024. [Online]. Available: <https://joern.readthedocs.io/en/latest/index.html>.
- [10] L. Szekeres, M. Payer, T. Wei, and D. Song, “Sok: Eternal war in memory,” in *2013 IEEE Symposium on Security and Privacy*, 2013, pp. 48–62. DOI: 10.1109/SP.2013.13.
- [11] J. D. Pereira, N. Ivaki, and M. Vieira, “Characterizing Buffer Overflow Vulnerabilities in Large C/C++ Projects,” *IEEE Access*, vol. 9, pp. 142 879–142 892, 2021. DOI: 10.1109/ACCESS.2021.3120349.
- [12] J. D. Pereira, J. H. Antunes, and M. Vieira, “A software vulnerability dataset of large open source c/c++ projects,” in *2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC)*, 2022, pp. 152–163. DOI: 10.1109/PRDC55274.2022.00029.
- [13] S. Shimmi, H. Okhravi, and M. Rahimi, *Ai-based software vulnerability detection: A systematic literature review*, 2025. arXiv: 2506.10280 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2506.10280>.
- [14] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, “Predicting vulnerable software components,” in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, Alexandria, Virginia, USA: ACM, 2007, pp. 529–540. DOI: 10.1145/1315245.1315311.
- [15] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995. DOI: 10.1007/BF00994018.
- [16] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [17] K. O’Shea and R. Nash, “An introduction to convolutional neural networks,” *arXiv preprint*, 2015, arXiv:1511.08458. [Online]. Available: <https://arxiv.org/abs/1511.08458>.
- [18] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, *A survey of machine learning for big code and naturalness*, 2018. arXiv: 1709.06182 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/1709.06182>.
- [19] J. Zhou et al., “Graph neural networks: A review of methods and applications,” *AI Open*, vol. 1, pp. 57–81, 2020. DOI: 10.1016/j.aiopen.2021.01.001.
- [20] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, “A Comprehensive Survey on Graph Neural Networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, Jan. 2021, ISSN: 2162-2388. DOI: 10.1109/tnnls.2020.2978386.

- [21] M. Zhang, Z. Cui, M. Neumann, and Y. Chen, "An End-to-End Deep Learning Architecture for Graph Classification," in *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, New Orleans, Louisiana, USA: AAAI Press, 2018, pp. 4438–4445. DOI: 10.1609/aaai.v32i1.11782.
- [22] N. S. Harzevili, J. Shin, J. Wang, S. Wang, and N. Nagappan, "Automatic static vulnerability detection for machine learning libraries: Are we there yet?" In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 2023, pp. 795–806. DOI: 10.1109/ISSRE59848.2023.00042.
- [23] S. Lipp, S. Banescu, and A. Pretschner, "An Empirical Study on the Effectiveness of Static C Code Analyzers for Vulnerability Detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, Virtual, South Korea: Association for Computing Machinery, 2022, pp. 544–555. DOI: 10.1145/3533767.3534380.
- [24] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, "Modeling and discovering vulnerabilities with code property graphs," in *2014 IEEE Symposium on Security and Privacy*, May 2014, pp. 590–604. DOI: 10.1109/SP.2014.44.
- [25] *Open hub, the open source network*, <https://openhub.net>, Accessed: 2026-04-22, 2023.
- [26] G. S P, S. Shankar, and V. Subramanian, "Machine learning approaches towards medical images," in Jan. 2023, pp. 124–145, ISBN: 9781668465233. DOI: 10.4018/978-1-6684-6523-3.ch006.
- [27] D. A. Wheeler, *Flawfinder: Static Analysis Tool for C/C++ Security Flaws*, <https://dwheeler.com/flawfinder/>, Accessed: 2025-07-09, 2023.
- [28] D. Marjamäki, *Cppcheck: A Tool for Static C/C++ Code Analysis*, <https://cppcheck.sourceforge.io/>, Accessed: 2025-07-09, 2016.
- [29] J. R. Henriques, J. D'Abruzzo Pereira, and M. Vieira, "Mining vulnerability and code repositories to study software security," in *Proceedings of the 13th Latin-American Symposium on Dependable and Secure Computing*, ser. LADC '24, Association for Computing Machinery, 2024, pp. 11–16, ISBN: 9798400717406. DOI: 10.1145/3697090.3697103.
- [30] MITRE Corporation, *Common Weakness Enumeration (CWE)*, <https://cwe.mitre.org/>, Accessed: 2025-08-02, 2025.
- [31] MITRE, *CWE-264: Permissions, Privileges, and Access Controls*, <https://cwe.mitre.org/data/definitions/264.html>, Accessed: 2026-02-06, 2006. Accessed: Jul. 19, 2006.
- [32] MITRE, *CWE-399: Resource Management Errors*, <https://cwe.mitre.org/data/definitions/399.html>, Accessed: 2026-02-06, 2006. Accessed: Jul. 19, 2006.
- [33] MITRE, *CWE-17: CWE CATEGORY: DEPRECATED: Code*, <https://cwe.mitre.org/data/definitions/17.html>, Accessed: 2026-02-06, 2006. Accessed: Jul. 19, 2006.
- [34] MITRE, *CWE-20: Improper Input Validation*, <https://cwe.mitre.org/data/definitions/20.html>, Accessed: 2026-02-06, 2006. Accessed: Jul. 19, 2006.
- [35] C. D. Xuan, "A new approach to software vulnerability detection based on CPG analysis," *Cogent Engineering*, vol. 10, no. 1, p. 2221962, 2023. DOI: 10.1080/23311916.2023.2221962.
- [36] J. Wang, H. Xiao, S. Zhong, and Y. Xiao, *Deepvulseeker: A novel vulnerability identification framework via code graph structure and pre-training mechanism*, 2022. arXiv: 2211.13097 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2211.13097>.
- [37] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks*, 2019. arXiv: 1909.03496 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/1909.03496>.
- [38] S. Cao et al., "Learning to detect memory-related vulnerabilities," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 2, Dec. 2023, ISSN: 1049-331X. DOI: 10.1145/3624744.
- [39] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "Mvd: Memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22, Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 1456–1468, ISBN: 9781450392211. DOI: 10.1145/3510003.3510219.
- [40] D. Hin, A. Kan, H. Chen, and M. A. Babar, *LineVD: Statement-level Vulnerability Detection using Graph Neural Networks*, 2022. arXiv: 2203.05181 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2203.05181>.
- [41] W. Zheng, Y. Jiang, and X. Su, *Vulspg: Vulnerability detection based on slice property graph representation learning*, 2021. arXiv: 2109.02527 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2109.02527>.
- [42] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 2365–2376. DOI: 10.1145/3510003.3510229.
- [43] M. Wang, X. Han, H. Zhang, Y. Guo, and J. Guo, "Dc-gar: Detecting vulnerabilities by utilizing graph properties and random walks to uncover richer features," *Automated Software Engineering*, vol. 32, no. 2, p. 61, 2025.
- [44] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, Jul. 2022, ISSN: 2160-9209. DOI: 10.1109/tdsc.2021.3051525.